

All of Statistics: A Concise Course in Statistical Inference
Solutions by David A. Lee

All errors, typographical and substantive, and other offenses, are entirely my own.

Chapter 7: Estimating the CDF and Statistical Functionals

Import Packages

Please see the associated GitHub repo for all code and comments to simulations.
[LINK HERE]

```
import numpy as np
from numpy.random import choice
import random
import matplotlib.pyplot as plt
import scienceplots
```

Question: 7.4.1

Prove Theorem 7.3.

Theorem (Wasserman 7.3)

At any fixed value of x ,

$$\begin{aligned}\mathbb{E}(\hat{F}_n(x)) &= F(x), \\ \mathbb{V}(\hat{F}_n(x)) &= \frac{F(x)(1-F(x))}{n}, \\ \text{MSE} &= \frac{F(x)(1-F(x))}{n} \rightarrow 0, \\ \hat{F}_n(x) &\xrightarrow{P} F(x).\end{aligned}$$

PROOF. $\mathbb{E}(\hat{F}_n(x)) = F(x)$:

By the definition of $\hat{F}_n(x)$, we have

$$\begin{aligned}\mathbb{E}(\hat{F}_n(x)) &= \mathbb{E}\left(\frac{\sum_{i=1}^n I(X_i \leq x)}{n}\right) \\ &= \frac{1}{n} \sum_{i=1}^n \mathbb{P}(X_i \leq x) \\ &= \frac{1}{n} (nF(x)) \\ &= F(x)\end{aligned}$$

$\mathbb{V}(\hat{F}_n(x)) = F(x)(1-F(x))/n$:

$$\begin{aligned}
 \mathbb{V}(\hat{F}_n(x)) &= \mathbb{E}(\hat{F}_n^2(x)) - \mathbb{E}(\hat{F}_n(x))^2 \\
 &= \mathbb{E}\left(\frac{1}{n^2} \left(\sum_{i=1}^n I(X_i \leq x)\right)^2\right) - F(x)^2 \\
 &= \frac{1}{n^2} \mathbb{E}\left(\sum_{j=1}^n \sum_{i=1}^n I(X_j \leq x) I(X_i \leq x)\right) - F(x)^2 \\
 &= \frac{1}{n^2} \left(\sum_{i=1}^n \mathbb{P}(X_i \leq x) + \sum_{j=1}^n \sum_{\substack{i=1 \\ i \neq j}}^n \mathbb{P}(X_j \leq x) \mathbb{P}(X_i \leq x)\right) - F(x)^2 \\
 &= \frac{1}{n^2} \left(nF(x) + n(n-1)F(x)^2\right) - F(x)^2 \\
 &= \frac{F(x) + (n-1)F(x)^2 - nF(x)^2}{n} \\
 &= \frac{F(x)(1-F(x))}{n}
 \end{aligned}$$

$$\text{MSE} = F(x)(1-F(x))/n \rightarrow 0:$$

$$\begin{aligned}
 \text{MSE} &= \text{bias}^2(\hat{F}_n(x)) + \mathbb{V}(\hat{F}_n(x)) \\
 &= \frac{F(x)(1-F(x))}{n} \rightarrow 0 \text{ as } n \rightarrow \infty
 \end{aligned}$$

$$\hat{F}_n(x) \xrightarrow{P} F(x) :$$

Since

$$\mathbb{E}(\hat{F}_n(x) - F(x))^2 = \mathbb{V}(\hat{F}_n(x)) \rightarrow 0 \text{ as } n \rightarrow \infty$$

It follows that $\hat{F}_n(x) \xrightarrow{qm} F(x)$, implying $\hat{F}_n(x) \xrightarrow{P} F(x)$. $\square$

Question: 7.4.2

Let $X_1, \dots, X_n \sim \text{Bernoulli}(p)$ and let $Y_1, \dots, Y_m \sim \text{Bernoulli}(q)$. Find the plug-in estimator and estimated standard error for p . Find an approximate 90 percent confidence interval for p . Find the plug-in estimator and estimated standard error for $p - q$. Find an approximate 90 percent confidence interval for $p - q$.

By Theorem 7.9,

$$\hat{p} = \int x d\hat{F}_n(x) = \boxed{\bar{X}_n}$$

$$\hat{s}e = \hat{\sigma}/\sqrt{n} = \boxed{\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}}$$

The 90 percent confidence interval for p has $z_{\alpha/2} = z_{.1/2} = 1.65$, so we have

$$\bar{X}_n \pm 1.65 \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}$$

For $p - q$, we have

$$\widehat{p - q} = \int x d\hat{F}_n(x) - \int y d\hat{F}_n(y) = \bar{X}_n - \bar{Y}_n$$

$$\hat{se} = \sqrt{\frac{\hat{\sigma}_p^2}{n} + \frac{\hat{\sigma}_q^2}{m}} = \sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{\hat{q}(1-\hat{q})}{m}}$$

$$90 \text{ percent CI} = \bar{X}_n - \bar{Y}_n \pm 1.65 \hat{se}$$

Question: 7.4.3

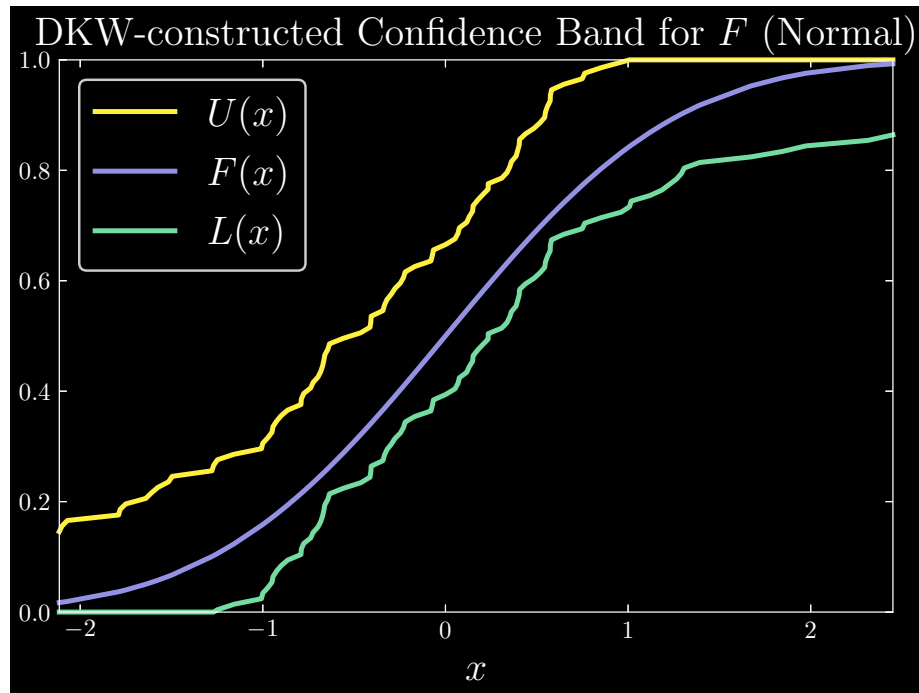
(Computer Experiment.) Generate 100 observations from a $N(0, 1)$ distribution. Compute a 95 percent confidence band for the CDF F . Repeat this 1000 times and see how often the confidence band contains the true distribution function. Repeat using data from a Cauchy distribution.

Using the DKW inequality, we can construct the confidence band from a standard normal distribution in the following manner:

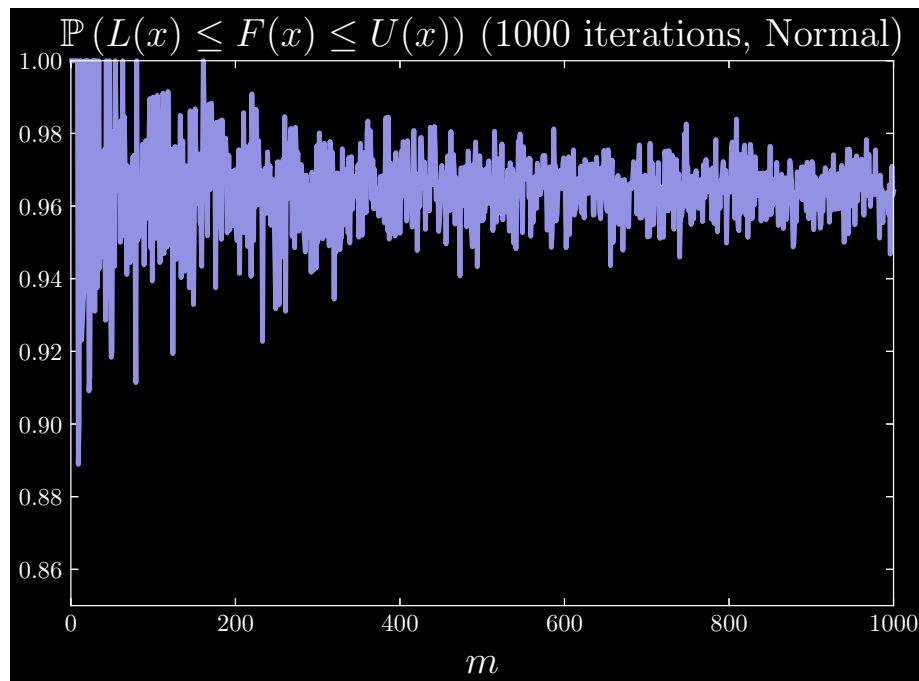
```
n = 100
alpha = 0.05
eps = np.sqrt((1/(2*n))*np.log(2/alpha))

def ecdfhat(sample):
    cdfhat = [ i/len(sample) for i in range(1, len(sample)+1)]
    return cdfhat;

def dkwintnormal():
    dkwtest, lowerbound, upperbound =
        np.empty(n), np.empty(n), np.empty(n)
    sample = np.sort(np.random.normal(0,1,n))
    ecdf = ecdfhat(sample)
    for i,j in zip(range(0, len(sample)), sample):
        lowerbound[i] = max(ecdf[i] - eps, 0)
        upperbound[i] = min(ecdf[i] + eps, 1)
        dkwtest[i] =
            (norm.cdf(j) >= lowerbound[i])
            & (norm.cdf(j) <= upperbound[i])
    return dkwtest, lowerbound, upperbound, ecdf, sample;
```



The probability that the CDF F is contained in the confidence band approaches 0.95 for large number of iterations:



Repeating the same for data from a standard Cauchy distribution, we get:

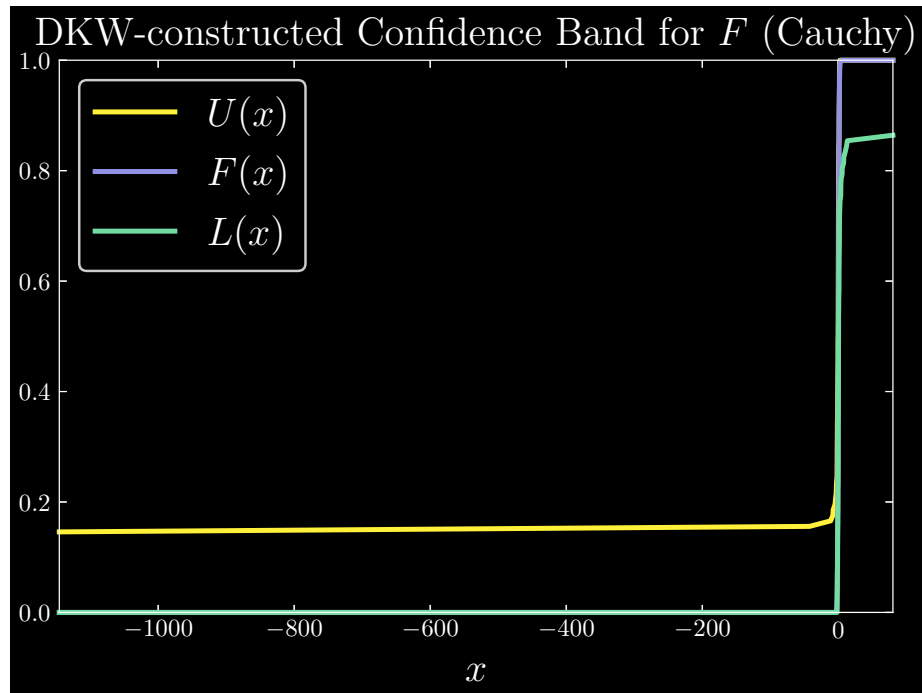
```

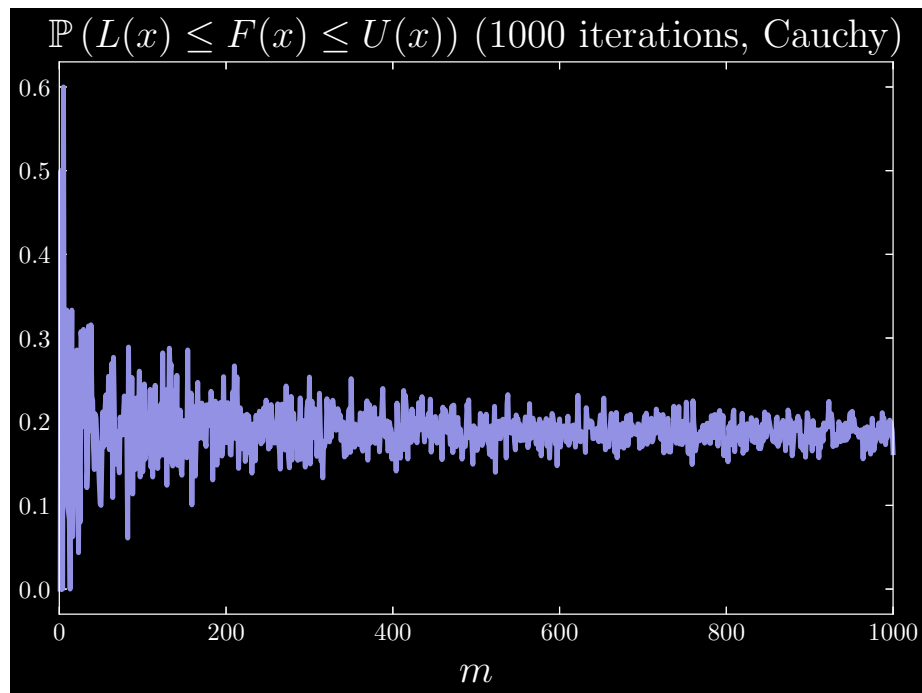
n = 100
alpha = 0.05
eps = np.sqrt((1/(2*n))*np.log(2/alpha))

def ecdfhat(sample):
    cdfhat = [ i/len(sample) for i in range(1, len(sample)+1)]
    return cdfhat;

def dkwintcauchy():
    dkwtest, lowerbound, upperbound =
        np.empty(n), np.empty(n), np.empty(n)
    sample = np.sort(np.random.standard_cauchy(n))
    ecdf = ecdfhat(sample)
    for i,j in zip(range(0, len(sample)), sample):
        lowerbound[i] = max(ecdf[i] - eps, 0)
        upperbound[i] = min(ecdf[i] + eps, 1)
        dkwtest[i] =
            (norm.cdf(j) >= lowerbound[i])
            & (norm.cdf(j) <= upperbound[i])
    return dkwtest, lowerbound, upperbound, ecdf, sample;

```





Question: 7.4.4

Let $X_1, \dots, X_n \sim F$ and let $\hat{F}_n(x)$ be the empirical distribution function. For a fixed x , use the central limit theorem to find the limiting distribution of $\hat{F}_n(x)$.

Using Theorem 7.3 and the proof from exercise 7.4.1, by the Central Limit Theorem, we have

$$\hat{F}_n(x) \approx N\left(F(x), \frac{F(x)(1-F(x))}{n}\right)$$

Question: 7.4.5

Let x and y be two distinct points. Find $\text{Cov}(\hat{F}_n(x), \hat{F}_n(y))$.

By definition of covariance, we have $\text{Cov}(\hat{F}_n(x), \hat{F}_n(y))$ equal to:

$$\begin{aligned}
 &= \mathbb{E} \left(\left(\hat{F}_n(x) - \mathbb{E}(\hat{F}_n(x)) \right) \left(\hat{F}_n(y) - \mathbb{E}(\hat{F}_n(y)) \right) \right) \\
 &= \mathbb{E} \left(\left(\hat{F}_n(x) - F(x) \right) \left(\hat{F}_n(y) - F(y) \right) \right) \\
 &= \mathbb{E} \left(\hat{F}_n(x) \hat{F}_n(y) - F(x) F(y) \right) \\
 &= \mathbb{E} \left(\frac{1}{n^2} \left(\sum_{i=1}^n I(X_i \leq x) \right) \left(\sum_{j=1}^n I(X_j \leq y) \right) \right) - F(x) F(y) \\
 &= \frac{1}{n^2} \mathbb{E} \left(\sum_{i=1}^n I(X_i \leq x) I(X_i \leq y) + \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n I(X_i \leq x) I(X_j \leq y) \right) \\
 &\quad - F(x) F(y)
 \end{aligned}$$

Now, for the first term to have non-zero terms in the sum, we must have $X_i \leq x, y$. Since x, y are distinct, either $x < y$ or $y < x$. Suppose without loss of generality that $x < y$. Then the first term resolves to

$$\mathbb{E} \left(\sum_{i=1}^n I(X_i \leq x) I(X_i \leq y) \right) = n \mathbb{P}(X_i \leq y) = nF(y)$$

For the second term, assuming independence, we have

$$\begin{aligned}
 \mathbb{E} \left(\sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n I(X_i \leq x) I(X_j \leq y) \right) &= n(n-1) \mathbb{P}(X_i \leq x) \mathbb{P}(X_j \leq y) \\
 &= n(n-1) F(x) F(y)
 \end{aligned}$$

Combining the terms, we can conclude that the covariance is equal to

$$\begin{aligned}
 &= \frac{1}{n^2} (nF(y) + n(n-1) F(x) F(y)) - F(x) F(y) \\
 &= \frac{F(y) + (n-1) F(x) F(y) - nF(x) F(y)}{n} \\
 &= \boxed{\frac{F(y)(1 - F(x))}{n}}
 \end{aligned}$$

Question: 7.4.6

Let $X_1, \dots, X_n \sim F$ and let $\hat{F}$ be the empirical distribution function. Let $a < b$ be fixed numbers and define $\theta = T(F) = F(b) - F(a)$. Let $\hat{\theta} = T(\hat{F}_n) = \hat{F}_n(b) - \hat{F}_n(a)$. Find the estimated standard error of $\hat{\theta}$. Find an expression for an approximate $1 - \alpha$ confidence interval for θ .

Using exercise 7.4.5, we can derive

$$\begin{aligned}\mathbb{V}(\hat{\theta}) &= \mathbb{V}(\hat{F}_n(b) - \hat{F}_n(a)) \\ &= \mathbb{V}(\hat{F}_n(b)) + \mathbb{V}(\hat{F}_n(a)) + 2 \text{Cov}(\hat{F}_n(b), \hat{F}_n(a)) \\ &= \frac{1}{n} (F(b)(1 - F(b)) + F(a)(1 - F(a)) + 2F(b)(1 - F(a))) \\ &= \frac{1}{n} (F(b) - F(a))(1 - (F(b) - F(a))) \\ \text{se}(\hat{\theta}) &= \sqrt{\mathbb{V}(\hat{\theta})} \\ \widehat{\text{se}}(\hat{\theta}) &= \sqrt{\frac{1}{n} (\hat{F}(b) - \hat{F}(a)) (1 - (\hat{F}(b) - \hat{F}(a)))}\end{aligned}$$

The $1 - \alpha$ confidence interval is given by

$$\hat{\theta} \pm z_{\alpha/2} \widehat{\text{se}}(\hat{\theta})$$

Question: 7.4.7

Data on the magnitudes of earthquakes near Fiji are available on the website for this book. Estimate the CDF $F(x)$. Compute and plot a 95 percent confidence envelope for F . Find an approximate 95 percent confidence interval for $F(4.9) - F(4.3)$.

The 95 percent confidence interval for $F(4.9) - F(4.3)$ is (0.495, 0.557). The DKW-constructed confidence band for F is as follows:

```
fijidata = pd.read_csv('fijiquakes.dat', delim_whitespace=True)

magdata = np.sort(fijidata['mag'])

n = len(magdata)
alpha = 0.05
eps = np.sqrt((1/(2*n))*np.log(2/alpha))

def ecdfhat(sample):
    cdfhat = [ i/len(sample) for i in range(1, len(sample)+1)]
    return cdfhat;
```

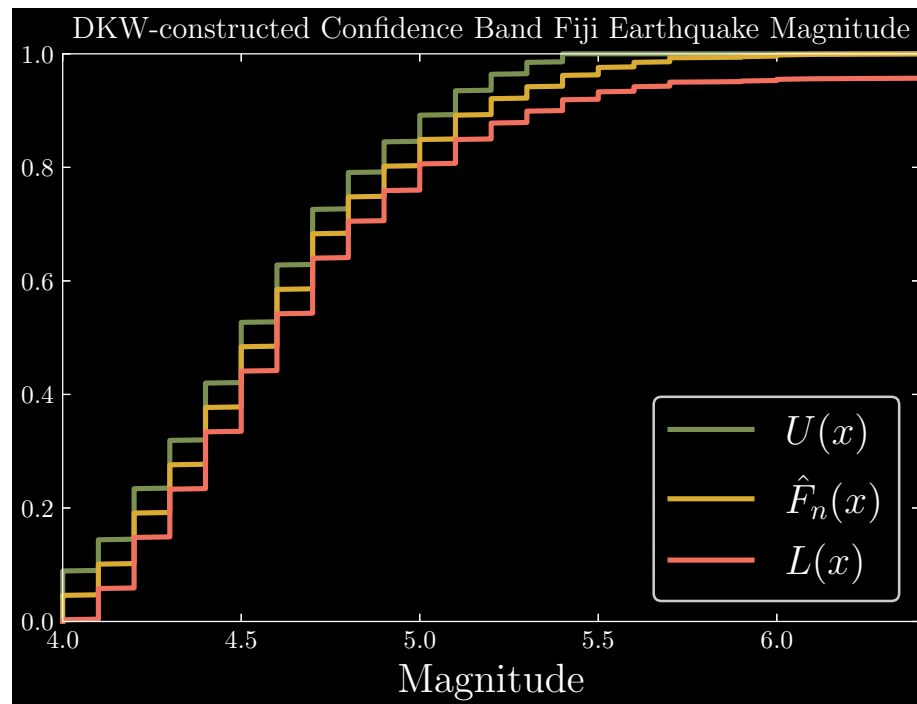
```

def dkwbounds():
    lowerbound, upperbound =
        np.empty(len(magdata)),
        np.empty(len(magdata))
    ecdf = ecdfhat(magdata)
    for i in range(0, len(magdata)):
        lowerbound[i] = max(ecdf[i] - eps, 0)
        upperbound[i] = min(ecdf[i] + eps, 1)
    return lowerbound, upperbound;

# Use results of exercise 7.4.6 for confidence interval

theta = ecdf(4.9) - ecdf(4.3)
sehat = np.sqrt((1/len(magdata))*theta*(1-theta))

```



Question: 7.4.8

Get the data on eruption times and waiting times between eruptions of the Old Faithful geyser from the website. Estimate the mean waiting time and give a standard error for the estimate. Also, give a 90 percent confidence interval for the mean waiting time. Now estimate the median waiting time. In the next chapter we will see how to get the standard error for the median.

The estimated mean, standard error, 90 percent confidence interval, and median are 70.897, 0.824, (69.537, 72.257), and 76.0, respectively.

```
faithful = pd.read_csv('faithful.dat', delim_whitespace=True)

waitingdata = faithful['waiting']

waitingmean = np.mean(waitingdata)
waitingse = np.std(waitingdata, ddof=1) / np.sqrt(np.size(waitingdata))

waitingci = (round(waitingmean - 1.65*waitingse, 3),
             round(waitingmean + 1.65*waitingse, 3))

waitingmedian = np.median(waitingdata)
```

Question: 7.4.9

100 people are given a standard antibiotic to treat an infection and another 100 are given a new antibiotic. In the first group, 90 people recover; in the second group, 85 people recover. Let p_1 be the probability of recovery under the standard treatment and let p_2 be the probability of recovery under the new treatment. We are interested in estimating $\theta = p_1 - p_2$. Provide an estimate, standard error, an 80 percent confidence interval, and a 95 percent confidence interval for θ .

We can model the odds of recovering as a Bernoulli variable. We estimate $\hat{\theta}$ as

$$\hat{\theta} = \widehat{p_1 - p_2} = \frac{90}{100} - \frac{85}{100} = \boxed{0.05}$$

The estimate of the standard error is

$$\begin{aligned} \widehat{\text{se}}(\hat{\theta}) &= \sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n} + \frac{\hat{p}_2(1-\hat{p}_2)}{n}} \\ &= \sqrt{\frac{90/100(1-90/100)}{100} + \frac{85/100(1-85/100)}{100}} \\ &\approx \boxed{0.0466} \end{aligned}$$

80 percent CI: $z_{\alpha/2} = z_{.2/2} = z_{0.1} \approx 1.28$

$$0.05 \pm 0.0597$$

95 percent CI: $z_{\alpha/2} = z_{0.05/2} = z_{0.025} \approx 1.96$

$$0.05 \pm 0.0914$$

Question: 7.4.10

In 1975, an experiment was conducted to see if cloud seeding produced rainfall. 26 clouds were seeded with silver nitrate and 26 were not. The decision to seed or not was made at random.

Let θ be the difference in the mean precipitation from the two groups. Estimate θ . Estimate the standard error of the estimate and produce a 95 percent confidence interval.

The estimates for θ , standard error, and the 95 percent confidence interval are 277.396, 136.124, and (10.593, 544.199), respectively.

```
clouds = pd.read_csv('clouds.dat', delim_whitespace=True)

unseeded = clouds['Unseeded_Clouds']
seeded = clouds['Seeded_Clouds']

diffmean = np.abs(np.mean(unseeded) - np.mean(seeded))
unseeded_se = np.std(unseeded) / np.sqrt(np.size(unseeded))
seeded_se = np.std(seeded) / np.sqrt(np.size(seeded))
diffse = np.sqrt(np.square(unseeded_se) + np.square(seeded_se))

diffci = (round(diffmean - 1.96*diffse, 3),
          round(diffmean + 1.96*diffse, 3))
```